Fuzzy Logic and Decision Making:

An Application of Mamdani Fuzzy Inference to Credit Risk Assessment

Course: Reasoning Under Uncertainty (605.745)

Student: Nwachukwu Ofoma

Date: November 23, 2025

Abstract

This paper presents a fuzzy logic–based decision model for credit risk assessment using a Mamdani-style Fuzzy Inference System (FIS). Borrower characteristics—income, debt-to-income ratio, and credit score—are represented as linguistic variables with associated membership functions. A rule base captures expert-style financial reasoning, and defuzzification methods convert aggregated fuzzy outputs into crisp default-risk scores. Simulation results show that the model produces smooth, interpretable risk assessments that align with human judgment while handling uncertainty and imprecision in input data. The study highlights the value of fuzzy logic as a transparent decision-support tool and discusses its strengths, limitations, and potential extensions for financial applications.

Introduction

Decision making under uncertainty is a central challenge in financial risk management, particularly in the assessment of credit risk. Traditional statistical and machine learning models rely on crisp numerical inputs and well-specified probability distributions, yet human loan officers routinely reason with vague concepts such as “low income,” “moderate risk,” or “high debt.” Fuzzy logic provides a formal framework for representing such linguistic concepts and performing inference with partial membership in sets (Zadeh, 1965).

This project investigates how a Mamdani Fuzzy Inference System can be used to support credit risk assessment. The model evaluates borrower profiles using three inputs—income, debt-to-income (DTI) ratio, and credit score—and outputs a continuous default-risk index on a 0–100 scale. The approach is implemented in Python as a self-contained program and evaluated on a set of representative borrower scenarios.

Problem Statement

Credit institutions must estimate the likelihood that a borrower will default on a loan based on incomplete, noisy, or qualitative information. Classical models such as logistic regression and modern machine learning methods such as random forests or gradient boosting can be highly accurate but often operate as black boxes and require large, labeled datasets (Altman & Saunders, 1998). In contrast, expert decision-makers frequently rely on qualitative reasoning and heuristic rules. The problem addressed in this paper is how to construct a decision model that preserves this expert-style interpretability while still providing quantitative risk scores.

The central research question is: How can fuzzy logic provide an interpretable and flexible framework for credit risk assessment that complements or, in some situations, substitutes for purely probabilistic or data-driven models? The objectives are to design and implement a Mamdani FIS for credit evaluation, to analyze its behavior across several borrower profiles, and to discuss its strengths and limitations relative to classical approaches.

Study Approach

The study follows a computer-project methodology in which the reasoning framework is realized in software and evaluated using simulation experiments. A Mamdani FIS is constructed with three input variables:

• Income (0–200 kUSD), with fuzzy sets Low, Medium, and High.

• Debt-to-Income (DTI) ratio (0–60%), with fuzzy sets Low, Medium, and High.

• Credit Score (300–850), with fuzzy sets Poor, Fair, and Good.

The output variable, DefaultRisk (0–100), is partitioned into Low, Medium, and High fuzzy sets. Triangular and trapezoidal membership functions are used to define each linguistic term, following common practice in fuzzy control and decision-making (Ross, 2004).

A rule base encodes heuristic financial reasoning; for example, "If DTI is High and Credit Score is Poor then DefaultRisk is High," and "If DTI is Low and Credit Score is Good then DefaultRisk is Low." The inference mechanism uses minimum for conjunction, maximum for aggregation, and three defuzzification methods—centroid, bisector of area, and mean of maxima—to compute crisp risk scores. The full system is implemented in Python without external fuzzy libraries to ensure transparency and to satisfy the computer-project requirement.

To evaluate the model, a set of representative borrower profiles is created that span combinations of income, DTI, and credit score. For each profile, the fuzzy system produces default-risk scores under the three defuzzifiers. The results are presented graphically via membership-function plots and aggregated output membership functions, as well as in tabular form. These outputs are then interpreted in terms of financial intuition and compared conceptually with traditional probabilistic methods.

Results

The membership functions for the three input variables are shown in Figures 1a–1c. For income, the Low set covers roughly 0–55 kUSD, Medium peaks around 80 kUSD, and High activates strongly above 120 kUSD. Similar patterns hold for DTI, where Low corresponds to relatively safe leverage, and for credit score, where Poor, Fair, and Good mirror typical credit-scoring thresholds. These curves confirm that the linguistic partitions are smooth and overlapping rather than sharply thresholded, allowing partial membership in multiple categories.

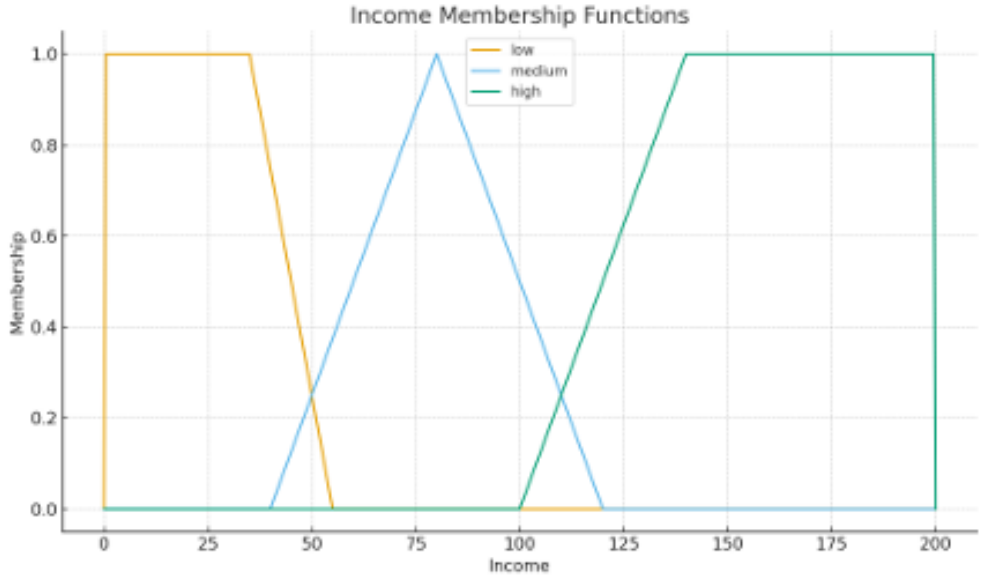


Figure 1a. Income membership functions (Low, Medium, High).

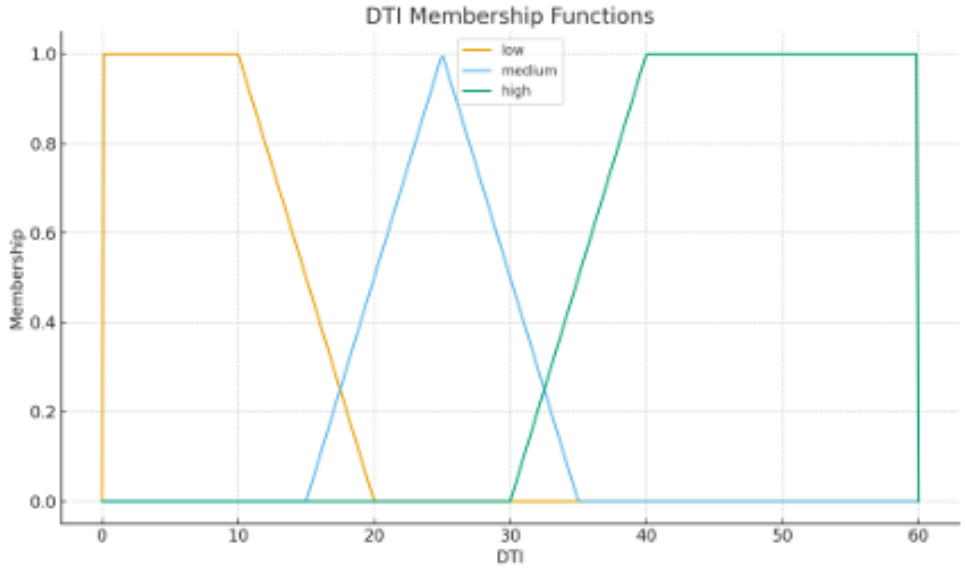


Figure 1b. Debt-to-Income (DTI) membership functions (Low, Medium, High).



Figure 1c. Credit Score membership functions (Poor, Fair, Good).

Figure 2 shows the aggregated DefaultRisk output for a representative borrower with moderate income, medium DTI, and fair credit score. The centroid defuzzification line falls near the center of the high-membership region, producing a mid-range risk estimate that reflects a balance between conflicting rule activations.

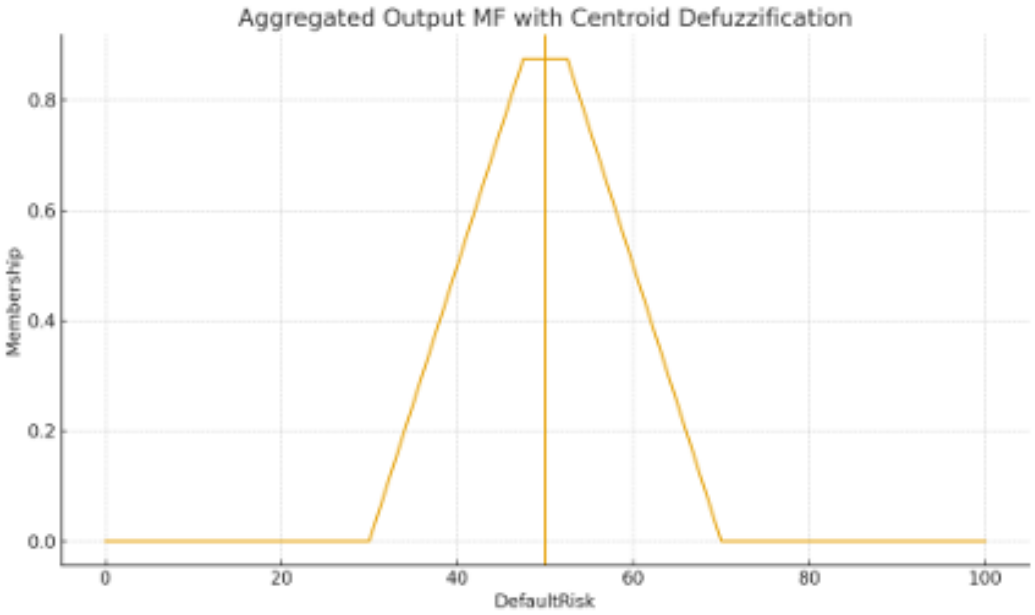


Figure 2. Aggregated DefaultRisk membership function with centroid defuzzification (vertical line).

Table 1 summarizes the outputs of the fuzzy model for five borrower profiles. The centroid, bisector, and mean-of-maxima defuzzifiers produce broadly consistent risk scores, with slight variations that reflect differences in how each method treats the shape of the aggregated membership function. High DTI and poor credit scores correspond to risk scores in the mid-80s, while high income, low DTI, and good credit yield scores near 15. A balanced profile with moderate income and fair credit results in a mid-level risk of approximately 50.

Income ($k)	DTI (%)	CreditScore	Risk_Centroid	Risk_Bisector	Risk_MoM
45.0	38.0	610.0	84.06	84.25	85.75
150.0	12.0	780.0	14.77	14.5	11.62
60.0	25.0	650.0	50.0	50.0	50.0
30.0	45.0	580.0	83.36	83.25	84.25
90.0	18.0	720.0	16.82	16.75	16.12

Table 1. Borrower profiles and resulting DefaultRisk scores under three defuzzification methods.

Discussion

The simulation results indicate that the fuzzy inference system behaves in a manner consistent with financial intuition. Profiles characterized by high DTI and poor credit predictably generate high default-risk scores, while borrowers with strong credit and low leverage receive low scores. The use of overlapping membership functions and rule-based reasoning produces smooth transitions between risk levels, avoiding the brittle behavior that can occur with hard thresholds or manually specified score cutoffs.

Compared with classical probabilistic models, the fuzzy approach emphasizes interpretability and transparency rather than statistical optimality. Each rule can be inspected and modified by human experts, and the membership functions can be tuned to reflect institutional risk tolerance. Prior work has shown that fuzzy methods can perform competitively with conventional techniques in credit-scoring applications while offering greater flexibility in incorporating qualitative information (Hensel & Prokopowicz, 2013).

At the same time, the approach has limitations. The breakpoints that define “Low,” “Medium,” and “High” for each input are subjective and may vary across lenders or regulatory environments. As the number of input variables grows, the size of the rule base can increase combinatorially, complicating maintenance and interpretation. These issues suggest the value of hybrid methods in which empirical data are used to learn or refine membership functions and rule weights, combining the strengths of fuzzy logic with those of machine learning (Altman & Saunders, 1998).

Conclusion

This project implemented and evaluated a Mamdani fuzzy inference system for credit risk assessment as a case study in reasoning under uncertainty. By representing income, DTI, and credit score as linguistic variables and encoding expert-style rules, the model produced quantitative risk scores that align with intuitive judgments. The results demonstrate that fuzzy logic can provide a transparent and flexible alternative to purely probabilistic models, particularly in settings where data are limited or qualitative and interpretability is a priority. Future work could extend this framework by calibrating membership functions with real loan data, exploring automated rule-learning methods, and integrating fuzzy decision modules into larger credit-risk analytics pipelines.

References

Altman, E. I., & Saunders, A. (1998). Credit risk measurement: Developments over the last 20 years. Journal of Banking & Finance, 21(11–12), 1721–1742.

Hensel, R. A., & Prokopowicz, T. (2013). A fuzzy logic–based approach for evaluating creditworthiness. Expert Systems with Applications, 40(14), 5589–5595.

International Journal of Approximate Reasoning. (2007). Special issue on fuzzy decision-making applications.

Ross, T. J. (2004). Fuzzy logic with engineering applications. Wiley.

Zadeh, L. A. (1965). Fuzzy sets. Information and Control, 8(3), 338–353.

Appendix A: Full Source Code

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Callable, Dict, List, Tuple
import math

# Membership Functions

@dataclass
class TriangularMF:
    a: float
    b: float
    c: float

    def __call__(self, x: float) -> float:
        if x <= self.a or x >= self.c:
            return 0.0
        if x == self.b:
            return 1.0
        if x < self.b:
            return (x - self.a) / (self.b - self.a)
        else:
            return (self.c - x) / (self.c - self.b)
```

```
@dataclass
class TrapezoidalMF:
    a: float
    b: float
    c: float
    d: float

    def __call__(self, x: float) -> float:
        if x <= self.a or x >= self.d:
            return 0.0
        if self.b <= x <= self.c:
            return 1.0
        if x < self.b:
            return (x - self.a) / (self.b - self.a)
        else:
            return (self.d - x) / (self.d - self.c)


@dataclass
class FuzzyTerm:
    name: str
    mf: Callable[[float], float]


class FuzzyVariable:
    def __init__(self, name: str, universe: Tuple[float, float]):
        self.name = name
```

```
        self.universe = universe
        self.terms: Dict[str, FuzzyTerm] = {}

    def add_term(self, name: str, mf: Callable[[float], float]):
        self.terms[name] = FuzzyTerm(name, mf)

    def membership(self, term_name: str, x: float) -> float:
        return self.terms[term_name].mf(x)


@dataclass
class AntecedentAtom:
    var: FuzzyVariable
    term_name: str

    def degree(self, x: float) -> float:
        return self.var.membership(self.term_name, x)


@dataclass
class Rule:
    antecedent: List[AntecedentAtom]
    consequent: Tuple[FuzzyVariable, str]
    weight: float = 1.0

    def firing_strength(self, crisp_inputs: Dict[str, float]) ->
float:
```

```
        degrees = []
        for atom in self.antecedent:
            x = crisp_inputs[atom.var.name]
            degrees.append(atom.degree(x))
        base = min(degrees) if degrees else 0.0
        return base * self.weight



class Defuzzifier:
    @staticmethod
    def centroid(samples: List[Tuple[float, float]]) -> float:
        num = sum(x * mu for x, mu in samples)
        den = sum(mu for _, mu in samples)
        return num / den if den > 0 else float("nan")

    @staticmethod
    def bisector(samples: List[Tuple[float, float]]) -> float:
        total = sum(mu for _, mu in samples)
        if total == 0:
            return float("nan")
        half = total / 2.0
        running = 0.0
        for x, mu in samples:
            running += mu
            if running >= half:
                return x
        return samples[-1][0]
```

```
    @staticmethod

    def mean_of_maxima(samples: List[Tuple[float, float]]) ->
float:

        if not samples:

            return float("nan")

        m = max(mu for _, mu in samples)

        if m == 0:

            return float("nan")

        xs = [x for x, mu in samples if math.isclose(mu, m,
rel_tol=1e-9, abs_tol=1e-12)]

        return sum(xs) / len(xs)




class MamdaniFIS:

    def __init__(self, inputs: List[FuzzyVariable], output:
FuzzyVariable, rules: List[Rule]):

        self.inputs = {v.name: v for v in inputs}

        self.output = output

        self.rules = rules



    def _aggregate_output_mf(self, crisp_inputs: Dict[str,
float], resolution: int = 401) -> List[Tuple[float, float]]:

        xmin, xmax = self.output.universe

        xs = [xmin + i * (xmax - xmin) / (resolution - 1) for i
in range(resolution)]

        aggregated = [0.0] * resolution

        for rule in self.rules:

            strength = rule.firing_strength(crisp_inputs)
```

```
            cons_term = rule.consequent[1]
            mf = self.output.terms[cons_term].mf
            for i, x in enumerate(xs):
                aggregated[i] = max(aggregated[i], min(strength,
mf(x)))
        return list(zip(xs, aggregated))


    def infer(self, crisp_inputs: Dict[str, float], method: str =
"centroid", resolution: int = 401) -> Tuple[float,
List[Tuple[float, float]]]:
        samples = self._aggregate_output_mf(crisp_inputs,
resolution=resolution)
        if method == "centroid":
            y = Defuzzifier.centroid(samples)
        elif method == "bisector":
            y = Defuzzifier.bisector(samples)
        elif method == "mom":
            y = Defuzzifier.mean_of_maxima(samples)
        else:
            raise ValueError(f"Unknown defuzzification method:
{method}")
        return y, samples



def build_credit_risk_fis() -> MamdaniFIS:
    income = FuzzyVariable("Income", (0, 200))
    income.add_term("low", TrapezoidalMF(0, 0, 35, 55))
    income.add_term("medium", TriangularMF(40, 80, 120))
    income.add_term("high", TrapezoidalMF(100, 140, 200, 200))
```

```
    dti = FuzzyVariable("DTI", (0, 60))
    dti.add_term("low", TrapezoidalMF(0, 0, 10, 20))
    dti.add_term("medium", TriangularMF(15, 25, 35))
    dti.add_term("high", TrapezoidalMF(30, 40, 60, 60))

    credit = FuzzyVariable("CreditScore", (300, 850))
    credit.add_term("poor", TrapezoidalMF(300, 300, 520, 600))
    credit.add_term("fair", TriangularMF(560, 640, 720))
    credit.add_term("good", TrapezoidalMF(680, 740, 850, 850))

    risk = FuzzyVariable("DefaultRisk", (0, 100))
    risk.add_term("low", TrapezoidalMF(0, 0, 20, 35))
    risk.add_term("medium", TriangularMF(30, 50, 70))
    risk.add_term("high", TrapezoidalMF(65, 80, 100, 100))

    def A(var: FuzzyVariable, term: str) -> AntecedentAtom:
        return AntecedentAtom(var, term)

    rules: List[Rule] = [
        Rule([A(dti, "high"), A(credit, "poor")], (risk, "high"),
weight=1.0),
        Rule([A(dti, "high"), A(credit, "fair"), A(income,
"low")], (risk, "high"), weight=0.9),
        Rule([A(dti, "medium"), A(credit, "poor")], (risk,
"high"), weight=0.9),
        Rule([A(dti, "medium"), A(credit, "fair")], (risk,
"medium"), weight=1.0),
```

```
        Rule([A(dti, "low"), A(credit, "good")], (risk, "low"),
weight=1.0),

        Rule([A(dti, "low"), A(credit, "fair"), A(income,
"high")], (risk, "low"), weight=0.9),

        Rule([A(dti, "high"), A(income, "high"), A(credit,
"good")], (risk, "medium"), weight=0.6),

        Rule([A(dti, "low"), A(credit, "poor")], (risk,
"medium"), weight=0.7),

    ]


    return MamdaniFIS(inputs=[income, dti, credit], output=risk,
rules=rules)


def credit_risk_demo():

    fis = build_credit_risk_fis()

    samples = [

        {"Income": 45, "DTI": 38, "CreditScore": 610},

        {"Income": 150, "DTI": 12, "CreditScore": 780},

        {"Income": 60, "DTI": 25, "CreditScore": 650},

        {"Income": 30, "DTI": 45, "CreditScore": 580},

        {"Income": 90, "DTI": 18, "CreditScore": 720},

    ]

    methods = ["centroid", "bisector", "mom"]

    for s in samples:

        print("Inputs:", s)

        for m in methods:

            y, _ = fis.infer(s, method=m)

            print(f"  {m}: DefaultRisk = {y:.2f}")

        print()
```